KEYSIGHT

TECHNOLOGIES

CloudLens Ansible

Azure Customer Runbook

Automated sensor deployment for Linux and Windows VMs at scale

Version	**v1.0.0**
Date	**June 2026**

Keysight Technologies | Network Visibility Solutions

Table of Contents

In Microsoft Word: right-click the entry below and choose 'Update Field' to populate the full table of contents.

Right-click and choose 'Update Field' to refresh the table of contents.

1. Executive Summary

CloudLens Ansible for Azure is a fully automated kit that deploys Keysight CloudLens sensors to every tagged virtual machine in an Azure subscription — Linux and Windows, from a single VM to 5,000+ — without any manual per-host steps.

It is built for customer DevOps and network teams who need network visibility instrumented quickly and reliably, and for Keysight Sales Engineers who need a repeatable, demoable proof-of-value asset.

Why it matters

- Eliminates the per-VM SSH / RDP grind. Tag a VM with cloudlens=yes and the kit handles the rest.
- Three entry points — Azure Portal one-click, Cloud Shell curl, or Docker — so customers run it wherever they already work.
- Production-tested against real Azure. End-to-end deployment of 3 mixed-OS sensors verified in 8 minutes.

Proof points

Scenario	Result	Time
Ubuntu 22.04 (private IP via jumpbox)	Sensor running	4 min
Ubuntu 22.04 (private IP via jumpbox)	Sensor running	4 min
Windows Server 2022 (WinRM direct)	Sensor running	6 min
CLMS 6.14.141 registration	All 3 sensors registered	<1 min
End-to-end	3/3 success	8 min

Date verified: 2026-06-02. Subscription: CloudLensPublic (eastus2).

2. Solution Overview

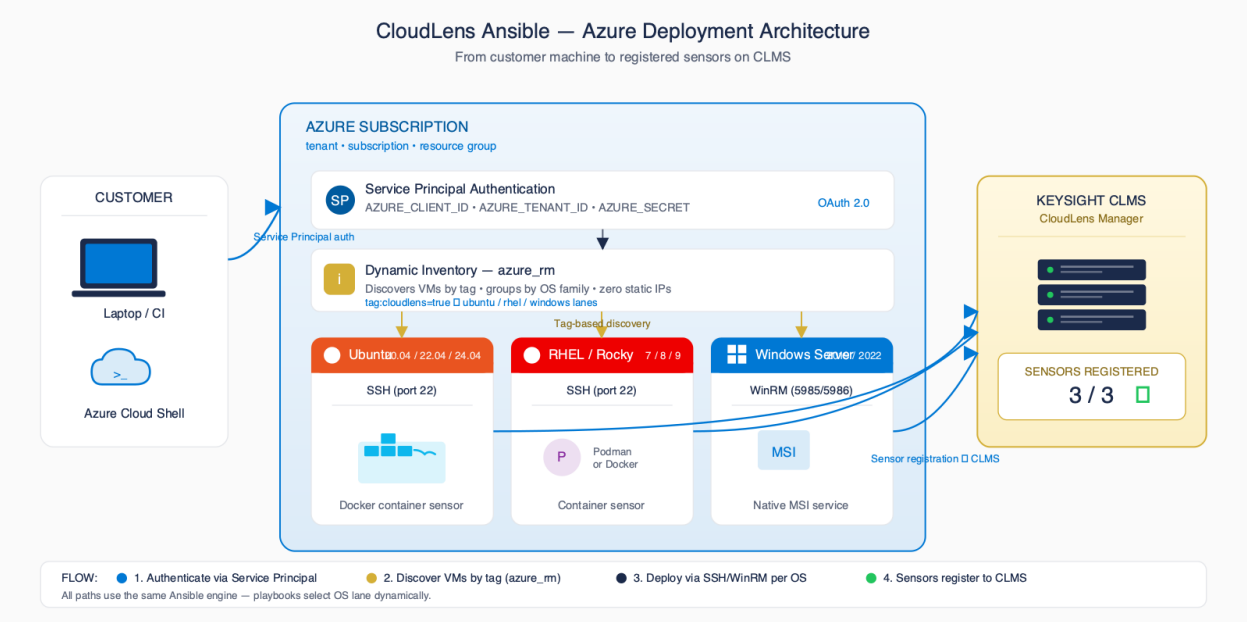

Figure 1 — End-to-end architecture: control point → Azure inventory → OS lanes → CLMS

A single Ansible control point authenticates to Azure (Service Principal, Managed Identity, or Cloud Shell session), queries the Azure Resource Manager API for every VM matching the customer's tag filter, and routes each host to the OS-specific playbook lane: Ubuntu, RHEL, or Windows.

Linux hosts run the CloudLens agent in a Docker container. Windows hosts run the CloudLens Windows sensor as a native service. Every sensor self-registers with CloudLens Manager (CLMS) on first start using the project key supplied in customer_input.yaml — no per-VM UI steps, no static inventory files to maintain.

3. Choosing Your Deployment Path

All three paths run the same Ansible engine — same playbooks, same automation. Pick the entry point that matches how your team works.

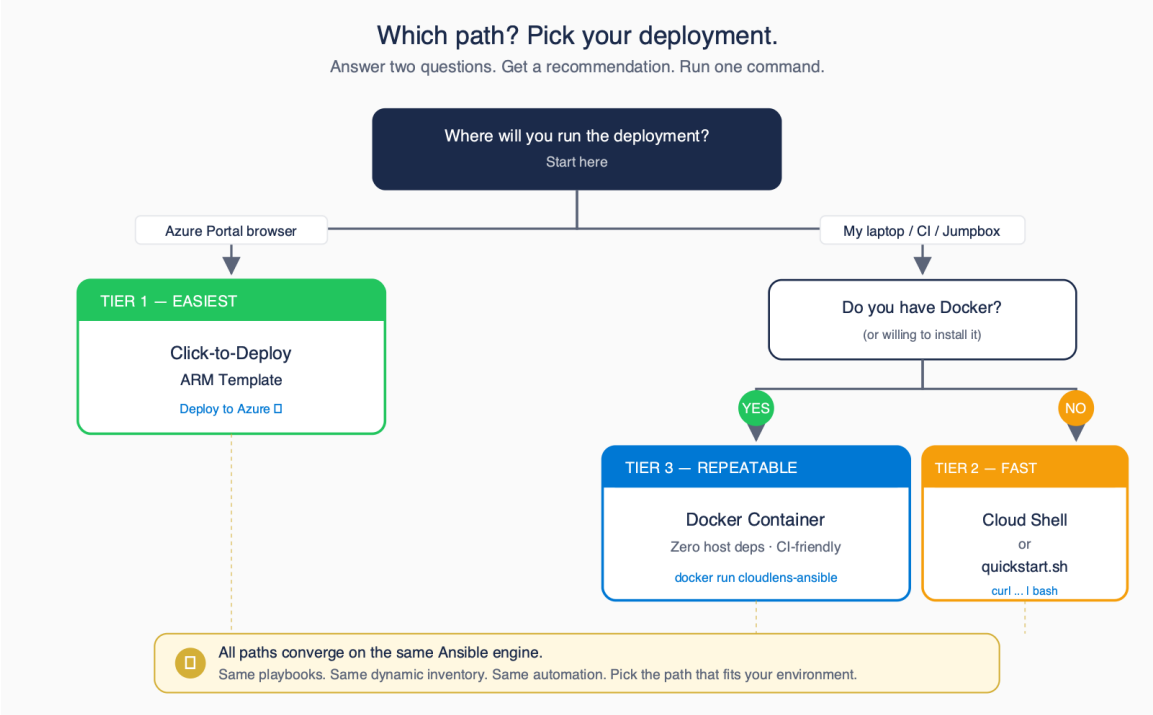

Figure 2 — Decision tree: pick the entry point that matches your environment

Tier	Best For	Tools Needed	Effort
Tier 1: One-Click Portal	Customers who live in the Azure Portal and want zero local setup	Web browser only	Lowest
Tier 2: Cloud Shell	Customers already authenticated to Azure in the browser	Azure Cloud Shell (pre-authenticated)	Low
Tier 3: Docker	Repeatable runs from a laptop or CI/CD pipeline	Docker, Service Principal, customer_input.yaml	Medium

4. Prerequisites Checklist (Printable)

Confirm every item below before kicking off a deployment. Tick each box as you go — the kit will not magically fix a missing prerequisite.

- ☐ Azure subscription with at least Reader + Virtual Machine Contributor roles
- ☐ Service Principal created (Tier 3 only — see scripts/setup_azure_sp.sh)
- ☐ CLMS Manager deployed and reachable from the target VM subnets
- ☐ Project Key obtained from the CLMS UI (Projects → API Keys)
- ☐ Target VMs tagged with cloudlens=yes, os=ubuntu|rhel|windows, env=prod|dev|qa
- ☐ SSH key (Linux) and Windows admin password ready in your control environment

Tip

Tags are how the dynamic inventory discovers hosts. If a VM is not tagged, the kit cannot see it — and that is the single most common support ticket. Use the bulk-tag script in Appendix B to tag an entire resource group in one shot.

5. Deployment — Step by Step

5.1 Tier 1: One-Click from the Azure Portal

Best when you want zero local tools. The Portal provisions a short-lived Ubuntu runner VM in your subscription, the runner uses Managed Identity to authenticate, deploys sensors to every tagged VM, then self-destructs.

1. Open the README in GitHub and click the 'Deploy to Azure' button.
2. Sign in to the Azure Portal when prompted; the ARM template opens in a 'Custom deployment' blade.
3. Fill in the four parameters: CLMS IP/FQDN, Project Key, Tag Filter (default cloudlens=yes), and Resource Group for the runner.
4. Review + Create. Azure provisions the runner VM (~2 minutes).
5. Watch the runner's boot diagnostics log — every tagged VM is sensored in parallel.
6. Runner self-destructs after 1 hour. Confirm sensors in CLMS → Sensors page.

[Screenshot placeholder — Azure Portal Deploy blade]

Expected outcome

Tagged VMs appear in CLMS → Sensors within 8 minutes. Runner VM is destroyed automatically. No artefacts left on the customer's laptop.

5.2 Tier 2: Azure Cloud Shell

Best when the customer is already logged into Azure in a browser tab. Cloud Shell is pre-authenticated, so there is no Service Principal to create and no local install.

7. Open Azure Cloud Shell (shell.azure.com) and choose Bash.
8. Run the quickstart bootstrap:
9. Answer the wizard prompts: CLMS IP, Project Key, tag filter, connection mode.
10. Confirm sensors in CLMS → Sensors page (~8 min for a small environment).

```
curl -sSL https://raw.githubusercontent.com/Keysight-Tech/cloudlens-ansible-azure/main/quickstart.sh | bash
```

Expected outcome

Wizard discovers tagged VMs, auto-tunes Ansible forks, deploys sensors, and prints a 3/3 success summary. All state lives in your Cloud Shell home directory — nothing installed locally.

5.3 Tier 3: Docker (laptop or CI/CD)

Best for repeatable runs from a developer laptop, a CI pipeline, or any container host. The image is pinned and hermetic — same result on macOS, Windows, Linux, GitHub Actions, GitLab CI, and Jenkins.

11. Create a Service Principal with `bash scripts/setup_azure_sp.sh` (one-time).
12. Copy customer_input.yaml.example to customer_input.yaml and fill in your CLMS IP, project key, and tag filter.
13. Export AZURE_SUBSCRIPTION_ID, AZURE_TENANT, AZURE_CLIENT_ID, AZURE_SECRET, ANSIBLE_WINRM_PASSWORD as env vars.
14. Run the container:

```
docker run --rm -it \
  -v $(pwd)/customer_input.yaml:/work/customer_input.yaml \
  -v $HOME/.ssh:/root/.ssh:ro \
  -e AZURE_SUBSCRIPTION_ID -e AZURE_TENANT \
  -e AZURE_CLIENT_ID -e AZURE_SECRET \
  -e ANSIBLE_WINRM_PASSWORD \
  ghcr.io/keysight-tech/cloudlens-ansible-azure:latest
```

Expected outcome

Container exits 0 with a 'sensors deployed: N/N' summary. Pin the image tag in CI to get hermetic, reproducible runs.

6. Supported VM Scenarios

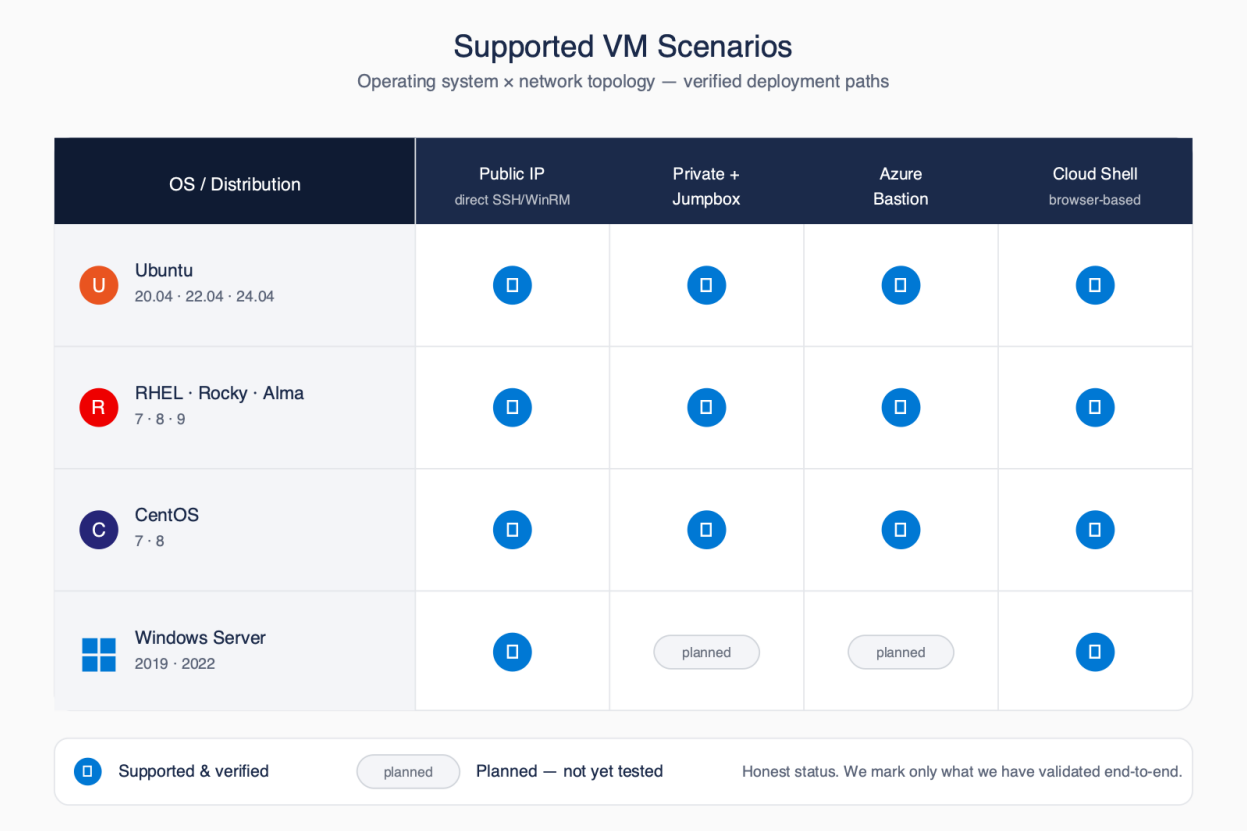

Figure 3 — VM compatibility matrix (OS × topology × auth method)

OS / Topology	Public IP direct	Private + Jumpbox	Azure Bastion	Cloud Shell
Ubuntu 20.04 / 22.04 / 24.04	Supported	Supported	Supported	Supported
RHEL 7 / 8 / 9	Supported	Supported	Supported	Supported
CentOS / Rocky / AlmaLinux	Supported	Supported	Supported	Supported
Windows Server 2019 / 2022	Supported	Planned	Planned	Supported

7. Verification Checklist (Printable)

After every deployment, walk this list. If any item fails, jump to section 9 (Troubleshooting Reference).

- ☐ All sensors visible in CLMS → Sensors page
- ☐ Filter by custom_tags (e.g. Customer=Acme) returns the expected VMs
- ☐ Test traffic from a sensored VM shows up in the destination tool/probe
- ☐ Container logs show no errors (Linux): `docker logs cloudlens-agent`
- ☐ CloudLens service running (Windows): `Get-Service CloudLensAgent`

8. Scaling Guide

VM Count	Auto Forks	Sharded?	Approx Time
1–50	20	No	5–10 min
50–500	50	No	15–30 min
500–2,000	200	No	30–60 min
2,000–10,000	500 / shard	Yes (auto)	30–60 min
10,000+	1000 / shard	AWX	1–2 hr

The kit auto-tunes the Ansible fork count based on the number of VMs the dynamic inventory discovers. Above 2,000 VMs it also auto-shards: the inventory is split into batches of `deploy.shard_size` VMs and the playbook runs them sequentially. This keeps memory and Azure API throttling under control while still finishing in under an hour for tens of thousands of hosts. See docs/SCALING.md in the repository for AWX/Tower integration details.

9. Troubleshooting Reference

Organised by symptom area. The full reference lives at docs/TROUBLESHOOTING.md in the repository.

Inventory

Symptom	Likely Cause	Fix
Inventory finds 0 VMs	Tags missing on VMs	az vm update --set tags.cloudlens=yes tags.os=<os> tags.env=prod
Subset of VMs missing	Tag value mismatch (case-sensitive)	Use exact lowercase values: ubuntu, rhel, windows, prod

SSH (Linux)

Symptom	Likely Cause	Fix
Permission denied (publickey)	Public key not present on target VM	Bootstrap via az vm run-command invoke or use jumpbox mode
Connection timeout	NSG blocks port 22 from control point	Switch connection.mode to jumpbox or bastion

WinRM (Windows)

Symptom	Likely Cause	Fix
WinRM timeout on 5985	WinRM disabled on Windows VM	Run playbooks/bootstrap_windows_winrm.yaml against the host
401 Unauthorized	ANSIBLE_WINRM_PASSWORD env var not exported	export ANSIBLE_WINRM_PASSWORD='...' before running the kit

Container (Linux sensor)

Symptom	Likely Cause	Fix
apt_pkg.Error: Signed-By conflict	Stale Docker apt source list	The playbook auto-cleans on the next run; rerun the kit
Container restarts in a loop	Wrong CLMS IP or project key	docker logs cloudlens-agent; fix manager_ip_or_fqdn / project_key

Registration with CLMS

Symptom	Likely Cause	Fix
Sensor not in CLMS UI	Wrong project key	Check CLMS → Projects → API Keys; update customer_input.yaml
Sensor offline after restart	CLMS IP unreachable from VM subnet	Open egress from VM subnet to CLMS on TCP/443

Appendix A — customer_input.yaml Schema

Full annotated example. Copy customer_input.yaml.example from the repository to customer_input.yaml and fill in values. Never commit customer_input.yaml to git — credentials belong in env vars.

```
# === Azure Environment ===
azure:
  subscription_id: "00000000-0000-0000-0000-000000000000"
  tenant_id:       "00000000-0000-0000-0000-000000000000"

  # Tag selector — VMs are matched by these tags
  tag_filters:
    cloudlens: "yes"
    env:       "prod"

  resource_groups: []   # empty = all RGs in the subscription
  locations: []         # empty = all regions

# === CloudLens Configuration ===
cloudlens:
  manager_ip_or_fqdn: "clms.customer.example.com"
  project_key:        "REPLACE_WITH_PROJECT_KEY"
  custom_tags:        "Env=Azure Region=eastus2 Customer=Acme"
  registry_type:      "insecure"   # or "secure"
  ssl_verify:         "no"
  auto_update:        "yes"
  local_ca_path:      "files/cloudlenscerts.crt"
  ca_cert_dir:        "/etc/ssl/certs"
  agent_container_name: "cloudlens-agent"
  log_max_size: "50m"
  log_max_file: "5"

# === Connection Mode ===
# direct_public | jumpbox | bastion | cloud_shell
connection:
  mode: "jumpbox"
  jumpbox_host:    "1.2.3.4"
  jumpbox_user:    "azureuser"
  jumpbox_ssh_key: "~/.ssh/id_rsa"
  bastion_resource_id: ""

# === Linux SSH ===
linux:
  ansible_user: "azureuser"
  ssh_key_file: "~/.ssh/id_rsa"

# === Windows WinRM ===
# Password MUST be set via env var: export ANSIBLE_WINRM_PASSWORD='...'
windows:
```

```
  ansible_user:                       "azureuser"
  ansible_connection:                 "winrm"
  ansible_winrm_transport:            "basic"
  ansible_winrm_server_cert_validation: "ignore"
  ansible_port:                       5985
  installer_path:     "files/cloudlens-win-sensor-6.13.0.359.exe"
  installer_filename: "cloudlens-win-sensor-6.13.0.359.exe"

# === Deployment Behavior ===
deploy:
  forks: 0                # 0 = auto-tune
  timeout_seconds: 60
  reinstall_if_unhealthy: true
  shard_size: 500         # for >2000 VMs
```

Appendix B — Bulk Tag Script

Apply the three required tags to every VM in a resource group, filtered by OS. Run these from Azure Cloud Shell or any shell with the Azure CLI logged in.

Ubuntu VMs

```
for vm in $(az vm list -g <RG> --query \
    "[?storageProfile.imageReference.offer=='0001-com-ubuntu-server-jammy'].name"
\
    -o tsv); do
  az vm update -g <RG> -n $vm \
    --set tags.cloudlens=yes tags.os=ubuntu tags.env=prod
done
```

RHEL VMs

```
for vm in $(az vm list -g <RG> --query \
    "[?storageProfile.imageReference.publisher=='RedHat'].name" \
    -o tsv); do
  az vm update -g <RG> -n $vm \
    --set tags.cloudlens=yes tags.os=rhel tags.env=prod
done
```

Windows VMs

```
for vm in $(az vm list -g <RG> --query \
    "[?storageProfile.osDisk.osType=='Windows'].name" \
    -o tsv); do
  az vm update -g <RG> -n $vm \
    --set tags.cloudlens=yes tags.os=windows tags.env=prod
done
```

Appendix C — Quick Links

Resource	Location
GitHub repository	https://github.com/Keysight-Tech/cloudlens-ansible-azure
README (technical entry point)	https://github.com/Keysight-Tech/cloudlens-ansible-azure/blob/main/README.md
Deployment guide	docs/DEPLOYMENT_GUIDE.md (in the repo)
Scaling guide	docs/SCALING.md (in the repo)
Troubleshooting guide	docs/TROUBLESHOOTING.md (in the repo)
Customer email template	docs/CUSTOMER_EMAIL.md (in the repo)
GitHub Issues (bug reports / feature requests)	https://github.com/Keysight-Tech/cloudlens-ansible-azure/issues
Related: vPB + Azure GWLB	https://github.com/Keysight-Tech/cloudlens-vpb-azure-gwlb

Support email subject (for the Keysight account team): 'CloudLens Ansible Azure — <customer name> — <brief issue>'.